

Análisis y Clasificación de Imágenes de Medios de Transporte

Implementación de Descriptores Visuales y Kernel RBF en Visión por Computador

Samuel David Duarte Rodríguez

Universidad Santo Tomás

Espacio Académico: Machine Vision

samuel.duarter@usantoto.edu.co

1. Base de Datos

Para el desarrollo de este proyecto se utilizó una base de datos obtenida desde la plataforma Kaggle orientada a la clasificación de medios de transporte. Este conjunto de datos fue seleccionado debido a la diversidad visual que presentan los objetos y a la posibilidad de aplicar técnicas de visión por computador para su análisis automático.

El dataset está compuesto por tres categorías principales: automóviles, aviones y barcos. Cada una de estas clases contiene un total de 700 imágenes, generando un conjunto global de 2100 imágenes. Esta distribución balanceada entre las clases permite evitar sesgos durante el proceso de entrenamiento del sistema de clasificación.

Con el fin de facilitar la organización de la información, se creó una carpeta principal denominada *dataset*. Dentro de esta carpeta se definieron tres subdirectorios correspondientes a cada categoría: *cars*, *planes* y *ships*. Esta estructura permite acceder fácilmente a las imágenes durante el procesamiento automático.

Las imágenes fueron nombradas de manera secuencial siguiendo el formato: Car (1).jpg, Plane (1).jpg y Ship (1).jpg. Este tipo de nomenclatura facilita la carga de archivos mediante programación y permite recorrer el conjunto de datos de forma ordenada.

Todas las imágenes se encuentran en formato JPG y fueron redimensionadas a una resolución uniforme de 300×300 píxeles. Este proceso de normalización espacial es importante para garantizar consistencia durante la extracción de características.

2. Selección del Kernel

El kernel utilizado en este trabajo corresponde a un Kernel de Base Radial (RBF), el cual es ampliamente utilizado en problemas de clasificación debido a su capacidad para modelar relaciones no lineales entre los datos. Este tipo de kernel permite medir la similitud entre vectores de características dentro de un espacio de alta dimensión.

Una de las principales razones para seleccionar el kernel RBF es su capacidad para proyectar los datos en un espacio donde las clases pueden separarse de manera más clara. En problemas de visión por computador, las características extraídas de las imágenes suelen presentar distribuciones complejas que no pueden separarse mediante modelos lineales simples.

Matemáticamente, el kernel RBF se define mediante la siguiente expresión:

$$K(x, c) = e^{-\gamma \|x - c\|^2}$$

donde x representa el vector de características de una imagen y c corresponde al centroide de una clase.

El parámetro γ controla la influencia de la distancia entre los vectores. Valores pequeños permiten considerar similitudes más globales mientras que valores grandes enfatizan similitudes locales entre las muestras.

La utilización de este kernel permite comparar cada imagen con los centroides de las clases mediante una medida de similitud continua, lo cual facilita el proceso de asignación de categorías.

3. Descriptores de Características

En los sistemas de visión por computador es necesario transformar las imágenes en representaciones numéricas que puedan ser analizadas por algoritmos de clasificación. Para este trabajo se emplearon tres descriptores principales: HOG, LBP y histogramas de color. Cada uno de estos descriptores actúa como un filtro de análisis que permite extraer información visual específica de la imagen.

El descriptor Histogram of Oriented Gradients (HOG) mide la orientación de los gradientes de intensidad presentes en la imagen. En términos prácticos, este descriptor identifica bordes, contornos y estructuras geométricas del objeto analizado. Esta información resulta especialmente útil porque permite capturar la forma global de los objetos. En el caso de este proyecto, HOG contribuye a diferenciar clases como automóviles, aviones y barcos debido a que cada uno presenta contornos y configuraciones estructurales distintas.

El descriptor Local Binary Pattern (LBP) mide patrones locales de textura mediante la comparación de la intensidad de cada píxel con la de sus vecinos cercanos. A partir de estas comparaciones se genera un patrón binario que describe variaciones de textura dentro de la imagen. Este descriptor es útil porque permite identificar características superficiales

presentes en los objetos o en el entorno donde aparecen, como superficies metálicas, agua o pavimento. En el contexto de este problema, LBP ayuda a diferenciar escenas donde predominan texturas asociadas al mar, al cielo o a entornos urbanos.

El histograma de color mide la distribución de los colores presentes en la imagen. Este descriptor cuantifica la frecuencia con la que aparecen determinados rangos de color dentro de la escena. Su importancia radica en que muchos objetos o contextos visuales presentan patrones cromáticos característicos. Por ejemplo, las imágenes de barcos suelen contener grandes regiones azules asociadas al agua, mientras que los aviones suelen aparecer en fondos dominados por el cielo. Esta información cromática contribuye a mejorar la separación entre las diferentes clases.

La combinación de estos tres descriptores permite generar una representación más completa de las imágenes, integrando información de forma, textura y color. Esta estrategia mejora la capacidad discriminativa del sistema, ya que cada descriptor aporta información complementaria que facilita la diferenciación entre las categorías de automóviles, aviones y barcos.

4. Entrenamiento y Clasificación

El proceso de entrenamiento inicia con la extracción de características de cada imagen utilizando los descriptores HOG, LBP y el histograma de color. Estos descriptores permiten obtener información sobre la forma, textura y distribución cromática de los objetos presentes en las imágenes. Los valores generados por cada descriptor se concatenan para formar un vector de características que representa numéricamente cada imagen analizada.

Una vez obtenidos los vectores de características, estos se organizan en una matriz de datos donde cada fila corresponde a una imagen y cada columna representa una característica específica. Posteriormente se aplica un proceso de normalización utilizando el método *StandardScaler*, el cual estandariza los datos para que tengan media cero y desviación estándar unitaria. A partir de estas características normalizadas se calculan los centroides de cada clase, obtenidos mediante el promedio de los vectores de características pertenecientes a cada categoría.

Durante la clasificación, cada imagen de prueba se procesa utilizando el mismo procedimiento de extracción de características y normalización. Posteriormente se calcula la similitud entre el vector de características de la imagen y los centroides de cada clase utilizando un kernel de base radial (RBF). La imagen se asigna a la clase cuyo centroide produce el mayor valor de similitud, permitiendo determinar automáticamente la categoría más probable entre automóviles, aviones y barcos.

5. Matriz de Confusión y Evaluación del Modelo

Para evaluar el desempeño del sistema de clasificación se construyó una matriz de confusión. Esta matriz permite analizar el número de clasificaciones correctas e incorrectas realizadas

por el modelo para cada categoría.

La matriz de confusión organiza las predicciones en filas correspondientes a las clases reales y columnas correspondientes a las clases predichas. De esta manera es posible identificar los casos en los que el sistema confunde una clase con otra.

A partir de esta matriz también se calcula el porcentaje de clasificación correcta o *accuracy*. Este indicador representa la proporción de imágenes correctamente clasificadas respecto al total de muestras evaluadas.

El accuracy se calcula mediante la siguiente expresión:

$$Accuracy = \frac{\sum_{i=1}^n M_{ii}}{\sum_{i=1}^n \sum_{j=1}^n M_{ij}}$$

El análisis de los resultados permite identificar errores de clasificación y evaluar la capacidad del sistema para diferenciar entre las distintas categorías de imágenes.

6. Código

A continuación se presenta el código desarrollado para la implementación del sistema de clasificación de imágenes basado en descriptores HOG, LBP, histogramas de color y un clasificador basado en centroides utilizando un Kernel de Base Radial (RBF).

7. Código Implementado

```
import numpy as np
import pandas as pd
import os
import cv2
import matplotlib.pyplot as plt
from pathlib import Path

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from skimage.feature import hog, local_binary_pattern

# =====
# VISUALIZAR IMAGENES DE ENTRENAMIENTO
# =====

def visualizar_categoria(carpeta, prefijo):
```

```

carpeta = Path("dataset") / carpeta

plt.figure(figsize=(12,12))
plt.suptitle(prefijo + "─Imágenes─de─entrenamiento", fontsize
            =18)

for i in range(1,201):

    ruta = carpeta / f"{prefijo}_{i}.jpg"

    img = cv2.imread(str(ruta))

    if img is None:
        continue

    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.subplot(20,10,i)

    plt.imshow(img)

    plt.axis("off")

plt.show()

# =====
# MOSTRAR CADA CATEGORIA
# =====

visualizar_categoria("cars","Car")
visualizar_categoria("planes","Plane")
visualizar_categoria("ships","Ship")

# =====
# CONFIGURACION
# =====

TRAIN_IMAGES = 200
TEST_IMAGES = 500

PATH_CARS = "dataset/cars"
PATH_PLANES = "dataset/planes"
PATH_SHIPS = "dataset/ships"

# =====

```

```

# DESCRIPTORES
# =====

def descriptor_hog(gray):

    return hog(
        gray,
        orientations=9,
        pixels_per_cell=(8,8),
        cells_per_block=(2,2),
        feature_vector=True
    )

def descriptor_lbp(gray):

    lbp = local_binary_pattern(gray,16,2,method='uniform')

    bins = int(lbp.max()+1)

    hist,_ = np.histogram(
        lbp.ravel(),
        bins=bins,
        range=(0,bins),
        density=True
    )

    return hist

def descriptor_color(img):

    hist = cv2.calcHist([img],[0,1,2],None
        ,[8,8,8],[0,256,0,256,0,256])

    hist = cv2.normalize(hist,hist).flatten()

    return hist

def descriptor_imagen(img):

    img = cv2.resize(img,(128,128))

    gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

    hog_f = descriptor_hog(gray)
    lbp_f = descriptor_lbp(gray)

```

```

        color_f = descriptor_color(img)

        return np.concatenate([hog_f, lbp_f, color_f])

# =====
# KERNEL RBF
# =====

def kernel_rbf(x, c):

    gamma = 0.002

    return np.exp(-gamma*np.linalg.norm(x-c)**2)

# =====
# CLASIFICADOR
# =====

def clasificar(x, centroids):

    scores = []

    for c in centroids:

        score = kernel_rbf(x, c)

        scores.append(score)

    return np.argmax(scores)

# =====
# CARGAR DATASET
# =====

def cargar_categoria(path, cantidad, inicio):

    archivos = sorted(os.listdir(path))[inicio:inicio+cantidad]

    data = []

    for f in archivos:

        img = cv2.imread(os.path.join(path, f))

        if img is not None:

```

```

        desc = descriptor_imagen(img)

        data.append(desc)

    return np.array(data)

# =====
# CARGAR DATABASE
# =====

print("Cargando entrenamiento...")

cars_train = cargar_categoria(PATH_CARS, TRAIN_IMAGES, 0)
planes_train = cargar_categoria(PATH_PLANES, TRAIN_IMAGES, 0)
ships_train = cargar_categoria(PATH_SHIPS, TRAIN_IMAGES, 0)

X_train = np.vstack((cars_train, planes_train, ships_train))

y_train = np.array(
    [0]*TRAIN_IMAGES +
    [1]*TRAIN_IMAGES +
    [2]*TRAIN_IMAGES
)

# =====
# NORMALIZACION
# =====

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

# =====
# CENTROIDES NORMALIZADOS
# =====

centroids = np.vstack([
    np.mean(X_train[y_train==0], axis=0),
    np.mean(X_train[y_train==1], axis=0),
    np.mean(X_train[y_train==2], axis=0)
])

print("Centroides normalizados")

```

```

print("Cars:", centroids[0][:10])
print("Planes:", centroids[1][:10])
print("Ships:", centroids[2][:10])

# =====
# CARGAR TEST
# =====

print("Cargando datos de prueba...")

cars_test = cargar_categoria(PATH_CARS, TEST_IMAGES, TRAIN_IMAGES)
planes_test = cargar_categoria(PATH_PLANES, TEST_IMAGES, TRAIN_IMAGES)
ships_test = cargar_categoria(PATH_SHIPS, TEST_IMAGES, TRAIN_IMAGES)

X_test = np.vstack((cars_test, planes_test, ships_test))

X_test = scaler.transform(X_test)

y_test = np.array(
    [0]*TEST_IMAGES +
    [1]*TEST_IMAGES +
    [2]*TEST_IMAGES
)

# =====
# CLASIFICACION
# =====

y_pred = []

for x in X_test:

    y_pred.append(clasificar(x, centroids))

y_pred = np.array(y_pred)

# =====
# MATRIZ DE CONFUSION
# =====

matriz = confusion_matrix(y_test, y_pred)

categorias = ["Cars", "Planes", "Ships"]

df = pd.DataFrame(matriz, index=categorias, columns=categorias)

```

```

print("MATRIZ_CONFUSION")

print(df)

# =====
# ACCURACY DEL CLASIFICADOR
# =====

correctos = np.trace(matriz)

total = np.sum(matriz)

accuracy = correctos / total * 100

print("Accuracy:", round(accuracy,2), "%")

```

8. Resultados Experimentales

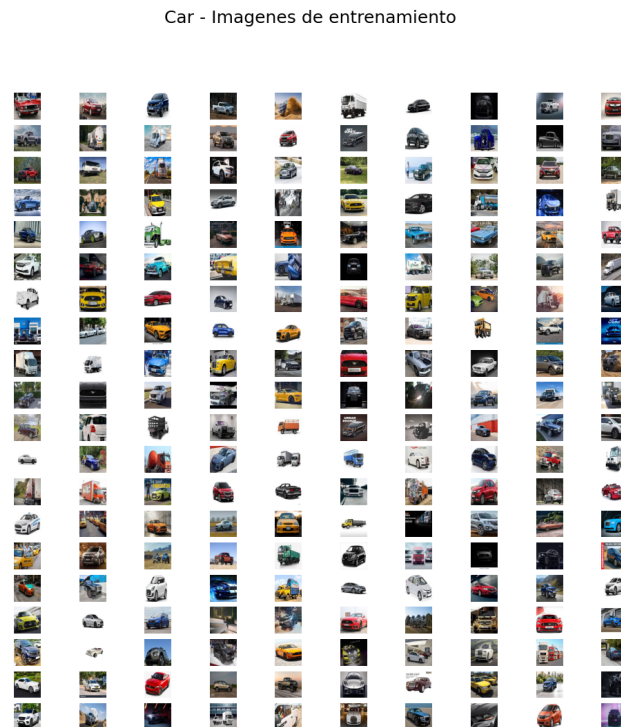


Figura 1: Visualización de las 200 imágenes utilizadas para el entrenamiento de la categoría Cars.

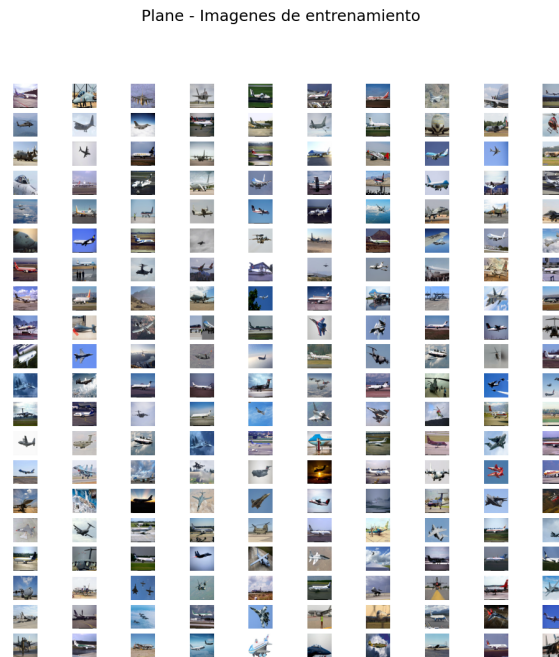


Figura 2: Visualización de las 200 imágenes utilizadas para el entrenamiento de la categoría Planes.

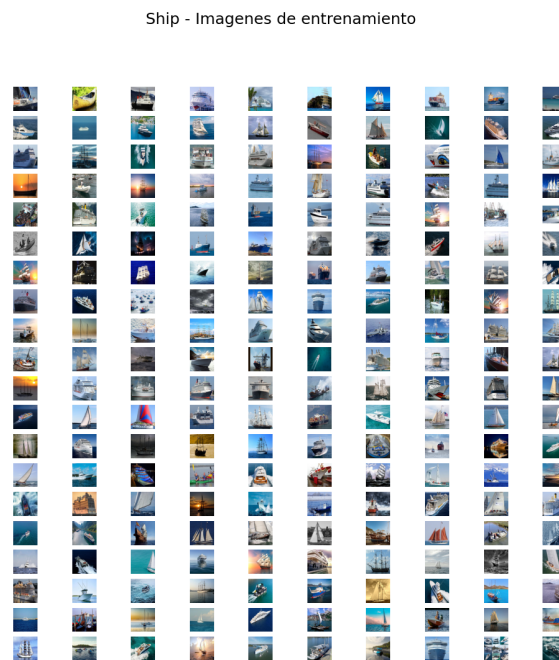


Figura 3: Visualización de las 200 imágenes utilizadas para el entrenamiento de la categoría Ships.

```

Centroides normalizados
Cars: [-0.18369895 -0.1625759 -0.24390018 -0.20374093 -0.31496174 -0.09224145
-0.16179335 -0.03412794 0.1987921 -0.27070128]
Planes: [ 0.26530726 0.19261197 0.19566943 0.09128273 0.06394165 -0.12730531
-0.03258034 -0.0328102 -0.13395054 0.20355225]
Ships: [-0.08160831 -0.03003607 0.04823074 0.11245821 0.25102009 0.21954675
0.1943737 0.06693814 -0.06484156 0.06714903]

Cargando datos de prueba...

MATRIZ CONFUSION
      Cars  Planes  Ships
Cars    329     55    116
Planes   33    361    106
Ships    42     70    388

MATRIZ CONFUSION (%)
      Cars  Planes  Ships
Cars    65.8    11.0   23.2
Planes   6.6    72.2   21.2
Ships     8.4    14.0   77.6

ACCURACY DEL CLASIFICADOR
Clasificaciones correctas: 1078
Total de muestras: 1500
Accuracy: 71.87 %

Clasificacion de imagen
Categoria predicha: Planes

```

Figura 4: Salida del sistema mostrando centroides normalizados, matriz de confusión, accuracy del modelo y clasificación de una imagen.

9. Explicación del Funcionamiento del Código

El código implementa un sistema de clasificación de imágenes utilizando herramientas de visión por computador en Python. Librerías como *NumPy* permiten trabajar con matrices numéricas, mientras que *OpenCV* se utiliza para leer y procesar las imágenes del conjunto de datos. El módulo *os* es importante porque permite acceder automáticamente a las carpetas del dataset y recorrer cada imagen sin necesidad de cargarlas manualmente. De esta forma, el programa organiza y prepara las imágenes para su posterior análisis, asegurando que el procesamiento sea sistemático y eficiente.

Posteriormente, cada imagen se transforma en un conjunto de características mediante descriptores visuales como HOG, LBP y el histograma de color. Estos descriptores permiten representar la forma, la textura y la distribución cromática de los objetos presentes en la imagen. Para comparar estas representaciones se utiliza un kernel de base radial (RBF), el cual mide la similitud entre vectores de características mediante una función exponencial basada en la distancia entre ellos. El uso de kernels es importante porque permite capturar relaciones no lineales entre los datos y mejorar la separación entre las diferentes clases.

Durante el entrenamiento se calculan centroides para cada categoría, los cuales corresponden al promedio de los vectores de características de cada clase. Estos centroides funcionan como puntos de referencia dentro del espacio de características para clasificar nuevas imágenes según su similitud. Finalmente, el rendimiento del modelo se analiza mediante una matriz de confusión, donde las filas representan las clases reales y las columnas las clases predichas. Los valores en la diagonal principal indican clasificaciones correctas, mientras que los valores fuera de la diagonal representan errores de clasificación entre las categorías.

10. Conclusiones

Los resultados obtenidos a partir de la matriz de confusión permiten analizar el desempeño general del sistema de clasificación implementado. Se observa que la diagonal principal, que representa las clasificaciones correctas, no supera el 80 % en ninguna de las categorías evaluadas. Este comportamiento puede estar asociado a factores como la variabilidad presente en las imágenes del conjunto de datos, la capacidad de discriminación de los descriptores utilizados o la forma en que el kernel mide la similitud entre las características extraídas.

Durante el desarrollo del trabajo se realizaron varios intentos para mejorar el desempeño del sistema, modificando los descriptores y ajustando los parámetros de los kernels con el objetivo de aumentar la precisión del modelo. Sin embargo, estos cambios no generaron mejoras significativas, ya que el porcentaje observado en la matriz de confusión permanecía prácticamente igual. Dentro de los resultados obtenidos, la categoría *Ships* presentó el mejor desempeño, mientras que la categoría *Cars* mostró un resultado menor, posiblemente debido a la mayor variabilidad de formas, ángulos y fondos presentes en las imágenes de automóviles.

Como posible mejora del sistema, se podrían explorar métodos alternativos de clasificación o estrategias más avanzadas en la extracción de características. Por ejemplo, al utilizar un clasificador basado en *Support Vector Machines (SVM)* se observó que la diagonal de la matriz de confusión podía superar el 80 % en varias categorías. Sin embargo, el objetivo del trabajo consistía en implementar el proceso de clasificación utilizando descriptores y kernels como mecanismo principal de comparación entre imágenes, por lo que el enfoque se mantuvo bajo este método.